Analyzing the Development Lifecycle of Vichan

Tristan Lopez, Noah Cash, Jonathan Lange, and Ar-Raniry Ar-Rasyid

Objective

Vichan is a free, open-source program that people use to run imageboard websites. Its code has been public on GitHub since 2010, and in 2026 the project officially came to an end. In this project, we want to look at the full history of changes made to Vichan and figure out how the work on it changed over time. We have two questions. First, as the project got older, did the developers stop adding new features and spend most of their time on security and bug fixes? Second, how much of that work was forced on them by outside problems, like newly found security holes, and how much was the team choosing to clean up its own code?

Motivation

A lot of open-source software is built by volunteers, and many of those projects slow down or stop entirely. We usually only see these projects at one point in time, either while they are growing or after they are gone. Vichan is different because its whole life, from the first change in 2010 to the end in 2026, is public. That gives us the chance to see what the work looked like at every stage, and whether the end of a project can be seen coming in its history. Going through thousands of changes by hand would be slow, and two people reading the same change might not agree on what it was. Sorting the changes automatically makes the question realistic to answer, as long as we check how accurate the sorting is.

Data

Our data comes from the official Vichan repository on GitHub (https://github.com/vichan-devel/vichan). A repository keeps every saved change to the code, which is called a commit. Each commit records who made the change, when they made it, a short message about it, and the exact lines of code that were added or removed. We will get this data through repository mining. We will download a full copy of the repository with Git, then write a Python script that pulls the history out into CSV files.

From a first download, the repository has about 9,600 commits. Those commits change files about 25,000 times and contain around 65,000 separate blocks of changed code. We will count each block of changed code as one record, so we expect to work with about 65,000 records.

We expect a few problems with this data. Many commit messages are vague, sometimes just "fix" or "update," so the message alone will not tell us what the change was for. To work around this, we will also look at which files and lines changed. The same developer also shows up under different names and emails, and one developer alone used at least six. If we do not combine these, one person will look like several. Finally, since our labels will be assigned automatically, some of them will be wrong, so we have to check them before we trust the results.

Approach

Our project focuses on the retrieval, cleaning, and analysis parts of the Digital Archaeology process. We will start by writing a Python script that downloads the repository's history and saves it as CSV files. Then we will clean the data. This means removing merge commits, which only combine work that is already in the history, putting all of the timestamps in one time zone, and combining the different names that belong to the same developer.

Next, we will sort each change into one of five categories: Feature, Bug Fix, Security Fix, Code Cleanup, or Documentation. We will start with keywords. For example, a message that mentions "XSS" or "vulnerability" will be labeled as a security fix. Anything the keywords cannot sort will be labeled by an AI language model. To see how accurate this is, we will label a small random sample by hand and compare it to the automatic labels. Last, we will count how many changes fall into each category per year and graph the results, which will show whether security and bug-fix work took up more of the project as it aged. Our main tools will be Git, Python, the pandas library, an AI language model, and Matplotlib for the graphs.

Team Responsibilities

Jonathan Lange will write the script that collects the repository data and saves it in a usable format. Tristan Lopez will clean the data, which includes removing merge commits and combining duplicate developer names. Ar-Raniry Ar-Rasyid will build the automatic sorting process and lead the accuracy check against the hand-labeled sample. Noah Cash will make the graphs and lead the writing of the final report.

Timeline and Milestones

Dates	Milestone
October 19	Team repository set up; preliminary commit statistics and activity timeline created
October 20	Initial Python extraction and analysis script written
October 22	Project proposal completed
October 23 – October 26	Full repository data, including changed code, collected and saved as CSV files
October 27 – October 30	Data cleaned and developer identities combined
October 31 – November 3	Classification tested on a small sample
November 4 – November 8	Full dataset classified and accuracy checked
November 9 – November 12	Analysis and graphs completed
November 13 – November 16	Final report and presentation completed
November 17	Project presentation

Expected Outcome

Our final result will be a data analysis, with graphs, that shows how the work on Vichan changed over its life. We expect it to show whether the project moved away from building new features and toward mostly security and maintenance work, and how much of its activity was a response to outside problems instead of the team improving its own code.